ЛР3. Поиск секретов: Gitleaks и pre-commit

Время: 2 академических часа. **Что получится:** вы найдёте секрет в файлах и в истории Git, уберёте его готовым исправлением, поставите автоматическую блокировку коммитов с секретами и оформите исключение для старой находки.

Перед началом: выполнена ЛР2, запущен Docker.

Коротко о главном

- **Секрет** — пароль, токен, ключ: строка, которая сама даёт доступ.
- **Git помнит всё.** Удалили секрет новым коммитом — в старом он остался.
- Секрет, попавший в репозиторий, считается **раскрытым**. Первое действие — **отозвать его и выпустить новый**. Удаление из кода — уже уборка.
- Gitleaks ищет двумя способами: по известным форматам (токены GitHub, AWS и т. д.) и по **энтропии** — «слишком случайная» строка рядом со словами `token`, `key`.
- **pre-commit** запускает проверку на вашем компьютере перед каждым коммитом. Его можно обойти, поэтому ту же проверку повторяют в CI.
- Токен в учебном проекте **фиктивный**. Работаем с ним как с настоящим.

Команда	Что проверяет
`gitleaks dir`	файлы в текущем виде
`gitleaks git`	всю историю коммитов

Параметр	Зачем
`--redact`	скрыть значение секрета в выводе, чтобы он не утёк через отчёт или журнал CI
`-v`	показать подробности каждой находки
`--report-path`	сохранить отчёт в файл

Ход работы

```
cd ~/devsecops-lab
source .venv/Scripts/activate      # Windows, Git Bash
# source .venv/bin/activate        # Linux и macOS
mkdir -p reports
```

Только Windows (Git Bash) — выполняйте в каждом новом окне терминала перед командами `docker run`:

```
export MSYS_NO_PATHCONV=1
```

Шаг 1. Проверить, что сканер запускается

```
docker run --rm ghcr.io/gitleaks/gitleaks:v8.24.2 version
```

Что увидите: `8.24.2`. Первый запуск скачивает образ.

Все сканеры с этой работы запускаются через Docker одинаково:

- `docker run --rm` — запустить и удалить контейнер после работы;
- `-v "$PWD:/src"` — показать контейнеру текущий каталог как `/src`;
- `ghcr.io/gitleaks/gitleaks:v8.24.2` — образ сканера **с точной версией**.

Если `Cannot connect to the Docker daemon` — не запущен Docker Desktop.

Шаг 2. Найти секрет в файлах

```
docker run --rm -v "$PWD:/src" ghcr.io/gitleaks/gitleaks:v8.24.2 dir --redact -v
 /src
echo "Код: $?"
```

Что увидите:

```
Finding:     DEMO_GITHUB_TOKEN = "REDACTED"
RuleID:      generic-api-key
Entropy:     4.020844
File:        /src/backend/config.py
Line:        3
Fingerprint: /src/backend/config.py:generic-api-key:3

WRN leaks found: 1
Код: 1
```

Код 1 — «найдены утечки». Это результат проверки, а не сбой.

Сработало правило `generic-api-key` (энтропия), а не правило для токенов GitHub: учебный токен короче настоящего.

В отчёт: вывод команды. Значение секрета в отчёт не копируйте.

Шаг 3. Найти секрет в истории

```
docker run --rm -v "$PWD:/src" ghcr.io/gitleaks/gitleaks:v8.24.2 git --redact -v
 --report-format json --report-path /src/reports/gitleaks-before.json /src
```

Что увидите: та же находка с новыми полями — `Commit`, `Author`, `Date`. Отпечаток (`Fingerprint`) теперь начинается с идентификатора коммита.

Заполните карточку находки:

Поле	Значение
Правило	
Файл и строка	
Коммит (первые 7 символов)	
Дата коммита	
Отпечаток	

В отчёт: карточка.

Шаг 4. Убрать секрет из кода

Готовое исправление переносит секрет из кода в переменные окружения:

```
cp -r ../secure-development-labs/solutions/lab3/. .
git diff
```

Что увидите: вместо строки с токеном — `os.getenv("DEMO_GITHUB_TOKEN", "")`. Значение теперь задаётся там, где приложение запускают, а в коде осталось только имя переменной.

Проверьте и сохраните:

```
pytest -q
docker run --rm -v "$PWD:/src" ghcr.io/gitleaks/gitleaks:v8.24.2 dir --redact -v
 /src
git add -A
git commit -m "lab3: read secrets from environment"
```

Что увидите: `7 passed`, затем `no leaks found`.

Если `gitleaks dir` нашёл что-то в папке `reports` — это отчёты других сканеров со случайными строками. В коммиты эта папка не попадает; такие находки здесь не учитывайте.

Шаг 5. Убедиться, что история помнит

```
docker run --rm -v "$PWD:/src" ghcr.io/gitleaks/gitleaks:v8.24.2 git --redact -v
 /src
git show HEAD~2:backend/config.py
```

Что увидите: `leaks found: 1`, а вторая команда печатает старую версию файла вместе с токеном. Файл чист, секрет доступен.

Опишите в отчёте порядок действий, будь токен настоящим. Заполните таблицу:

№	Действие	Кто выполняет	Зачем
1	Отозвать токен	владелец учётной записи	украденная копия перестаёт работать
2	Выпустить новый и передать приложению через переменные окружения		
3	Проверить журналы использования токена		
4	Убрать токен из кода		
5	Поставить автоматическую проверку		

В отчёт: вывод команд, таблица.

Шаг 6. Поставить блокировку коммитов

В проекте уже есть файл настроек `.pre-commit-config.yaml`:

```yaml
repos:
  - repo: https://github.com/gitleaks/gitleaks
    rev: v8.24.2
    hooks:
      - id: gitleaks
```

`repo` — откуда взять проверку, `rev` — точная версия, `id` — какая проверка.

```
pre-commit install
pre-commit run --all-files
```

Что увидите:

```
pre-commit installed at .git/hooks/pre-commit
Detect hardcoded secrets.................................................Passed
```

Первый запуск занимает несколько минут: скачивается и собирается Gitleaks.

Шаг 7. Проверить, что блокировка работает

Попробуйте закоммитить файл с «секретом»:

```
printf 'token=ghp_8ae31cacf141669ddfb5da7e31c8a2f3a4\n' > secret-demo.txt
git add secret-demo.txt
git commit -m "Test secret blocking"
```

Что увидите:

```
Detect hardcoded secrets.................................................Failed
- hook id: gitleaks
- exit code: 1
...
File:        secret-demo.txt
```

Коммит не создан. Уберите файл:

```
git log --oneline
git restore --staged secret-demo.txt
rm secret-demo.txt
```

В отчёт: вывод заблокированного коммита и `git log --oneline` (коммита `Test secret blocking` в нём нет).

Шаг 8. Оформить исключение для старой находки

Сейчас `gitleaks git` всегда завершается с ошибкой из-за первого коммита. В конвейере это означало бы вечно «красную» проверку — на такую перестают смотреть и пропускают новые находки.

Находка разобрана: токен учебный, из кода удалён. Это основание для исключения. Получите отпечаток:

```
docker run --rm -v "$PWD:/src" ghcr.io/gitleaks/gitleaks:v8.24.2 git --redact -v
 /src | grep Fingerprint
```

Создайте в корне проекта текстовый файл `.gitleaksignore` из двух строк: комментарий с обоснованием и отпечаток **из вашего вывода** (у каждого студента он свой):

```
# Учебный токен из первого коммита: фиктивный, из кода удалён в ЛР3. <Фамилия>,
<дата>
162fb607f51d8fda36ca6ba640f692d4eb9bac0c:backend/config.py:generic-api-key:3
```

Проверьте и сохраните:

```
docker run --rm -v "$PWD:/src" ghcr.io/gitleaks/gitleaks:v8.24.2 git --redact -v
 /src
git add -A
git commit -m "lab3: document historical finding in .gitleaksignore"
```

Что увидите: `no leaks found`; при коммите хук показал `Passed`.

Если по-прежнему `leaks found: 1` — отпечаток скопирован неточно: он должен совпадать до символа, без пробелов по краям.

Правила исключений:

- исключается **одна находка по отпечатку**, а не правило и не файл целиком;
- рядом — обоснование, автор, дата;
- действующий настоящий секрет в исключения не вносят — его отзывают.

В отчёт: содержимое `.gitleaksignore`.

Шаг 9. Чего Gitleaks не нашёл

Откройте `backend/config.py`. В нём осталось значение по умолчанию:

```
FLASK_SECRET_KEY = os.getenv("FLASK_SECRET_KEY", "dev-only-change-me")
```

Этим ключом приложение подписывает cookie сессии. Gitleaks о нём молчит: строка не похожа на случайную. Запишите в отчёт, почему общеизвестный ключ подписи — проблема и почему сканер его пропустил.

Заодно проверьте Semgrep:

```
semgrep scan --config .semgrep.yml --metrics=off --error .
```

Что увидите: `0 findings` — последняя находка из ЛР2 ушла.

Шаг 10. Как это встраивается в CI

В конвейере ЛР5 будет та же команда `gitleaks git`:

```yaml
- uses: actions/checkout@v4
  with:
    fetch-depth: 0          # получить всю историю, а не один последний коммит
- name: Scan Git history
  run: gitleaks git --redact --no-banner .
```

Где проверка	Плюс	Минус
pre-commit на ноутбуке	секрет не покидает компьютер	можно не установить или обойти (`--no-verify`)
CI на сервере	обойти нельзя, проверяется вся история	секрет к этому моменту уже отправлен

Нужны обе: первая — для скорости, вторая — для гарантии.

Отчёт

1. Вывод `gitleaks dir` и `gitleaks git` до исправления.
2. Карточка находки.
3. Результаты после исправления.
4. Доказательство, что секрет остался в истории; таблица реагирования.
5. Вывод заблокированного коммита.
6. Файл `.gitleaksignore`.
7. Что Gitleaks пропустил и почему.

Контрольные вопросы

1. Почему удаление токена новым коммитом не устраняет утечку?
2. Чем `dir` отличается от `git`?
3. Зачем нужен `--redact`?
4. Почему локального хука недостаточно?
5. Когда находку можно внести в `.gitleaksignore`, а когда нельзя?

Оценивание

Критерий	Баллы
Сканирование файлов и истории, карточка находки	25
Проверка исправления, порядок реагирования	20
pre-commit установлен, блокировка доказана	25
Обоснованное исключение	15
Пропуск сканера, контрольные вопросы	15
Итого	**100**